# OOPS Combined Practice Questions

Medium Level Java Problems for Interview Preparation

| | |
|---|---|
| Practice Goal | Students must analyze classes, relationships, encapsulation needs, overriding points, and parent-reference polymorphism before coding. |
| Level | Medium - suitable after learning all four OOPS principles. |
| Instruction | Do not directly jump into code. First write classes, variables, methods, parent-child relationship, and expected output flow. |

Questions included: Smart Vehicle Management System, Online Payment System, Smart Employee Management System.

# Question 1 - Smart Vehicle Management System

## Problem Statement

Create a Java program for a Smart Vehicle Management System. The system should manage different vehicles and show how each vehicle starts its engine differently.

## Requirements

Create an Abstract Parent Class

- Create an abstract class named Vehicle.
- This class should represent common details shared by all vehicles.

Add Private Variables

- Add these private variables inside Vehicle: vehicleNumber, brand, speed.

Constructor

- Create a constructor to initialize vehicleNumber, brand, and speed.

Encapsulation

- Create getters and setters for all private variables.

Abstract Method

- Create an abstract method named startEngine().
- This method should be implemented differently by child classes.

Display Method

- Create a method named displayDetails() to display vehicle number, brand, and speed.

Create Child Classes

- Create two child classes: Car and Bike.
- Both should extend Vehicle.

Method Overriding

- Override startEngine() in both child classes.
- Car and Bike should have different engine start messages.

Main Method

- Create multiple vehicle objects.
- Use parent class reference.
- Call displayDetails() and overridden startEngine() methods.

## Concepts Covered

- Encapsulation - private variables with getters and setters
- Abstraction - abstract class and abstract method
- Inheritance - Car and Bike inherit Vehicle
- Method Overriding - different startEngine() implementations
- Runtime Polymorphism - parent reference holding child objects
- Constructors - initializing object data

### Sample Input

```
Car Details:
Vehicle Number : KL07AB1234
Brand          : Toyota
```

```
Speed          : 120

Bike Details:
Vehicle Number : KL08CD5678
Brand          : Yamaha
Speed          : 90
```

### Sample Output

```
Vehicle Number : KL07AB1234
Brand          : Toyota
Speed          : 120
Car engine starts using push button system.

---------------------------------

Vehicle Number : KL08CD5678
Brand          : Yamaha
Speed          : 90
Bike engine starts using self-start system.
```

# Question 2 - Online Payment System

## Problem Statement

Design a Java program for an Online Payment System where different payment methods process payments in different ways.

## Requirements

### Create an Interface

- Create an interface named Payment.
- Inside it, declare a method named makePayment(double amount).

### Create Payment Classes

- Create three classes: UPIPayment, CardPayment, and NetBanking.
- Each class should implement the Payment interface.

### Method Implementation

- Implement makePayment(double amount) differently in each payment class.
- Each class should print a different success message.

### Create Customer Class

- Create a class named Customer.
- Add private variables: customerName and accountBalance.

### Constructor

- Create a constructor to initialize customerName and accountBalance.

### Encapsulation

- Create getters and setters for customerName and accountBalance.

### Display Method

- Create displayCustomer() method to show customer details.

### Main Method

- Create one customer object.
- Create different payment method objects.
- Use interface reference to call makePayment().

## Concepts Covered

- Interface - Payment contract
- Abstraction - hiding payment implementation details
- Encapsulation - customer data protected using private variables
- Polymorphism - interface reference calls different implementations
- Loose Coupling - main method depends on Payment interface, not specific classes

### Sample Input

```
Customer Details:
Customer Name   : Reshma
Account Balance : 25000.0

Payments:
UPI Payment     : 1500.0
Card Payment    : 5000.0
Net Banking     : 3000.0
```

### Sample Output

```
Customer Name   : Reshma
Account Balance : 25000.0

UPI Payment of 1500.0 successful.

---------------------------------

Card Payment of 5000.0 successful.

---------------------------------

Net Banking Payment of 3000.0 successful.
```

# Question 3 - Smart Employee Management System

## Problem Statement

Create a Java program for managing employees in a company. The system should calculate bonuses differently for different employee roles.

## Requirements

### Create a Parent Class

- Create a parent class named Employee.

### Add Private Variables

- Add these private variables inside Employee: employeeId, employeeName, salary.

### Constructor

- Create a constructor to initialize employeeId, employeeName, and salary.

### Encapsulation

- Create getters and setters for all private variables.

### Create Method

- Create a method named calculateBonus() inside the parent class.

### Create Child Classes

- Create two child classes: Developer and Manager.
- Both classes should inherit from Employee.

### Method Overriding

- Override calculateBonus() differently in both child classes.
- Developer bonus and Manager bonus should be calculated differently.

### Display Method

- Create a method named displayEmployeeDetails().
- Display employee ID, employee name, salary, and calculated bonus.

### Main Method

- Create multiple employee objects.
- Use parent class reference.
- Call overridden methods and display details.

## Concepts Covered

- Encapsulation - employee fields are private
- Inheritance - Developer and Manager inherit Employee
- Method Overriding - different bonus logic
- Runtime Polymorphism - parent reference calls child method
- Constructors - initialize employee details

### Sample Input

```
Developer:
Employee ID   : 101
Employee Name : Arjun
Salary        : 60000

Manager:
```

```
Employee ID   : 102
Employee Name : Priya
Salary        : 85000
```

### Sample Output

```
Employee ID   : 101
Employee Name : Arjun
Salary        : 60000.0
Developer Bonus : 12000.0

---------------------------------

Employee ID   : 102
Employee Name : Priya
Salary        : 85000.0
Manager Bonus : 25000.0
```

# Student Submission Checklist

Before submitting the solution, students must verify the following points.

- Class names are meaningful and match the question.
- Private variables are used wherever encapsulation is required.
- Constructors are used to initialize object data.
- Getters and setters are written correctly.
- Inheritance relationship is correct.
- Overridden methods are actually called using parent/interface reference.
- Output format is clean and readable.
- No unnecessary code is added.